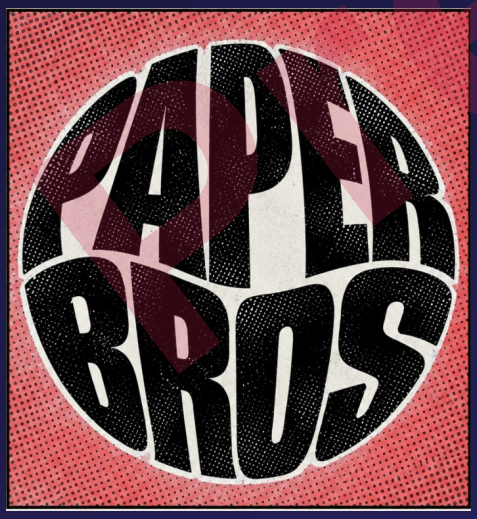
PREVIEW

CAMBRIDGE O LEVEL · SYLLABUS 2210 · 2026 – 2028

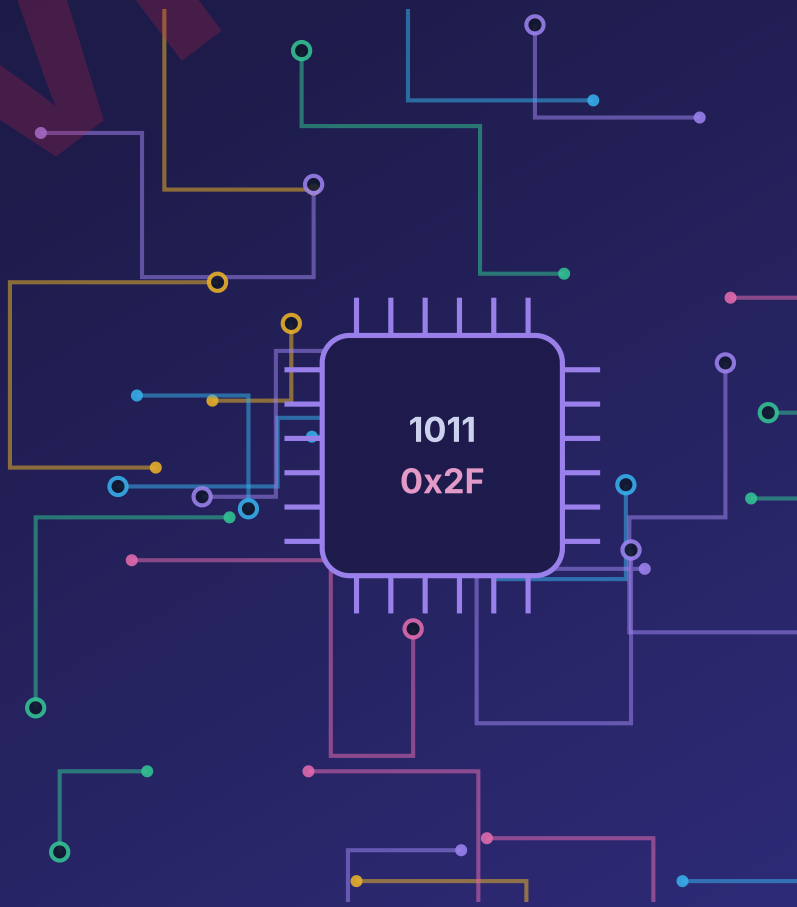
Computer Science

Complete study notes · diagrams · worked examples · exam tips · practice questions with answers

1 · Data Representation
2 · Data Transmission
3 · Hardware
4 · Software
5 · The Internet
6 · Emerging Tech
7 · Algorithms
8 · Programming
9 · Databases
10 · Boolean Logic



DEVELOPED BY STUDENTS,
FOR STUDENTS.



Paper 1 · Computer Systems | Paper 2 · Algorithms, Programming and Logic

Includes pseudocode reference & 15-mark scenario practice

A Product of PaperBros

PREVIEW

Chapter Page Index

Cambridge O Level Computer Science 2210 · Complete Study Notes

SECTION	TITLE	PAPER	STARTS ON	PAGES
Chapter 1	Data Representation	Paper 1	3	3–11
Chapter 2	Data Transmission	Paper 1	12	12–20
Chapter 3	Hardware	Paper 1	21	21–33
Chapter 4	Software	Paper 1	34	34–40
Chapter 5	The Internet and its Uses	Paper 1	41	41–47
Chapter 6	Automated and Emerging Technologies	Paper 1	48	48–54
Chapter 7	Algorithm Design and Problem-Solving	Paper 2	55	55–65
Chapter 8	Programming	Paper 2	66	66–75
Chapter 9	Databases	Paper 2	76	76–80
Chapter 10	Boolean Logic	Paper 2	81	81–86
Reference	Pseudocode Quick Reference	Paper 2	87	87–88
Exam practice	15-Mark Scenario Question & Exam Technique	Paper 2	89	89–94

Page numbers refer to the numbers printed at the bottom of each page. Each chapter ends with its practice questions and answers.

Preview

PREVIEW

Contents

Cambridge O Level Computer Science 2210 · syllabus for examination in 2026, 2027 and 2028

CHAPTER 1 · PAPER 1 · P.3
Data Representation
Number systems · hex · binary addition · shifts · two's complement · text, sound, images · file sizes · compression

CHAPTER 2 · PAPER 1 · P.12
Data Transmission
Packets · packet switching · serial/parallel · duplex · USB · parity · checksum · check digits · ARQ · encryption

CHAPTER 3 · PAPER 1 · P.21
Hardware
CPU · FDE cycle · cores, cache, clock · embedded systems · I/O devices · sensors · storage · virtual memory · cloud · network hardware

CHAPTER 4 · PAPER 1 · P.34
Software
System & application software · OS functions · firmware · interrupts · high/low-level languages · translators · IDEs

CHAPTER 5 · PAPER 1 · P.41
The Internet and its Uses
Internet vs WWW · URLs · HTTP/HTTPS · browsers · DNS · cookies · digital currency · blockchain · cyber security

CHAPTER 6 · PAPER 1 · P.48
Automated & Emerging Technologies
Automated systems · robotics · artificial intelligence · expert systems · machine learning

CHAPTER 7 · PAPER 2 · P.55
Algorithm Design & Problem-Solving
PDLC · decomposition · structure diagrams · flowcharts · searching & sorting · validation · test data · trace tables

CHAPTER 8 · PAPER 2 · P.66
Programming
Variables · data types · selection · iteration · strings · operators · subroutines · library routines · arrays · files

CHAPTER 9 · PAPER 2 · P.76
Databases
Fields & records · data types · primary keys · validation · SQL queries

CHAPTER 10 · PAPER 2 · P.81
Boolean Logic
Logic gates · truth tables · logic circuits · logic expressions

REFERENCE · P.87
Pseudocode Quick Reference
Every construct in Cambridge format with its Python equivalent

EXAM PRACTICE · P.89
Scenario Question & Exam Technique
Two full 15-mark scenarios with model answers · command words · checklist

How the exam works

1 PAPER 1 – COMPUTER SYSTEMS

- 1 hour 45 minutes · **75 marks · 50%**
- Short-answer and structured questions on **Topics 1–6**
- All questions compulsory · **no calculator**

2 PAPER 2 – ALGORITHMS, PROGRAMMING AND LOGIC

- 1 hour 45 minutes · **75 marks · 50%**
- Topics **7–10** + a **15-mark scenario** question
- Code answers in **pseudocode** (scenario: pseudocode, Python, VB or Java) · **no calculator**

Assessment objective	What it tests	Whole O Level	Paper 1	Paper 2
AO1	Knowledge and understanding of principles and concepts	40%	60%	20%
AO2	Applying knowledge to a given context, incl. analysing and designing programs	40%	20%	60%
AO3	Evaluating systems, making reasoned judgements, presenting conclusions	20%	20%	20%

Preview

PREVIEW

How to use these notes

KEY DEFINITIONS
Exact definitions and facts you must learn – these earn marks directly.

EXAM TIPS
How questions are asked and marked, and how to pick up every mark.

COMMON MISTAKES
Traps that students fall into every year.

WORKED EXAMPLES
Step-by-step methods – cover the answer and try it yourself first.

BEYOND THE SYLLABUS
Useful extras and shortcuts that help understanding (not required for the exam).

PRACTICE QUESTIONS
Each chapter ends with quick-fire and O Level style structured questions, followed by answers and mark guidance.

These notes cover every learning outcome in the syllabus. The best preparation is to combine them with **Cambridge past papers and mark schemes** and with regular **programming practice** on a computer.

Maths toolkit for O Level Computer Science

Calculators are **not allowed** in either paper. The syllabus expects you to add, subtract, multiply and divide; use averages, random numbers, decimals, fractions, percentages and ratios; use positive and negative integers and real numbers; use arithmetic and logical operators; use binary, denary and hexadecimal; and use counting, totalling and rounding. Learn these by heart:

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
2^n	1	2	4	8	16	32	64	128	256	512	1024	2048	4096	8192	16384	32768	65536

HANDY FACTS

- n bits → 2^n values, range 0 to $2^n - 1$
- 8 bits: 0–255 · two's complement: −128 to +127
- $16^2 = 256 \cdot 16^3 = 4096$
- 1 KiB = 1024 B = 2^{10} B · 1 MiB = 2^{20} B · 1 GiB = 2^{30} B
- bits ÷ 8 = bytes · bytes ÷ 1024 = KiB
- 1 hex digit = 4 bits · 2 hex digits = 1 byte

MENTAL-ARITHMETIC TRICKS

- × 1024 = × 1000 + × 24 (e.g. 3 × 1024 = 3000 + 72 = 3072)
- Divide by a power of two by **halving repeatedly** (÷ 8 = halve 3 times)
- Cancel powers of two before multiplying out: 2048 × 1024 × 8 ÷ 8 ÷ 1024 ÷ 1024 = 2048 ÷ 1024 = 2
- Average = total ÷ count; percentage = part ÷ whole × 100
- MOD = remainder, DIV = whole-number quotient: 17 = 5 × 3 + 2 → DIV(17, 5) = 3, MOD(17, 5) = 2

Preview

PREVIEW

CHAPTER 1 · SYLLABUS TOPIC 1

Data Representation

Paper 1 · Computer Systems

By the end of this chapter you can:

- Explain why computers use binary
- Convert between denary, binary and hexadecimal
- Explain why hexadecimal is used, with examples
- Add two 8-bit binary numbers and spot overflow
- Perform logical left and right shifts
- Use 8-bit two's complement for negative numbers
- Explain how text, sound and images are stored
- Use bit, nibble, byte, KiB … EiB correctly
- Calculate image and sound file sizes
- Compare lossy and lossless compression (incl. RLE)

1.1 Number systems

Why computers use binary

A computer is built from billions of tiny electronic switches called **transistors**. Each switch has only two states: **on** (1) or **off** (0). Two states are easy to make, easy to tell apart and very reliable, so all data – numbers, text, sound, images, video and program instructions – must be converted into binary before the computer can process it. The data is processed using **logic gates** and held temporarily in **registers**.

KEY DEFINITIONS

- **Denary** – base 10 system; uses digits 0–9; place values are powers of 10.
- **Binary** – base 2 system; uses digits 0 and 1; place values are powers of 2.
- **Hexadecimal (hex)** – base 16 system; uses 0–9 and A–F (A=10 … F=15); place values are powers of 16.
- **Bit** – a single binary digit (0 or 1). **Nibble** = 4 bits. **Byte** = 8 bits.



64 + 32 + 8 + 1 = **105**

Figure 1.1 *8-bit binary place values. Add the values of the columns that contain a 1.*

Converting binary → denary

Write the place values above each bit and **add the place values wherever there is a 1**.

WORKED EXAMPLE – 10110101 TO DENARY

128	64	32	16	8	4	2	1
1	0	1	1	0	1	0	1

128 + 32 + 16 + 4 + 1 = **181**

Preview

PREVIEW

Converting denary → binary

METHOD A – SUBTRACT PLACE VALUES

Convert **200**. Go left to right: does the place value fit?

- 128 fits → 1, remainder 72
- 64 fits → 1, remainder 8
- 32 no → 0 · 16 no → 0
- 8 fits → 1, remainder 0
- 4, 2, 1 → 0, 0, 0

200 = 11001000

METHOD B – DIVIDE BY 2

Keep dividing by 2 and record remainders; read **bottom → top**.

÷2	Result	Rem
200	100	0
100	50	0
50	25	0
25	12	1
12	6	0
6	3	0
3	1	1
1	0	1

Read upward: **11001000**

EXAM TIP

Always write the answer with the number of bits the question asks for (e.g. 8-bit → `00001101`, not `1101`). The syllabus allows binary numbers of up to **16 bits**; the 16-bit place values continue 256, 512, 1024, 2048, 4096, 8192, 16384, 32768. The largest 8-bit value is **255**; the largest 16-bit value is **65 535**. In general, *n* bits give 2^n different values (0 to $2^n - 1$).

Hexadecimal

Denary	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

Learn this table by heart – every hex digit is exactly one **nibble** (4 bits). That is the whole secret of hex ↔ binary conversion.

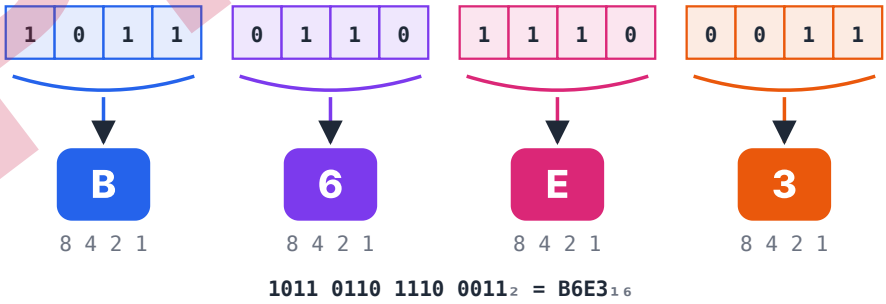


Figure 1.2 *Split binary into nibbles from the* ***right****; each nibble becomes one hex digit (and vice versa). Pad with leading 0s if needed.*

HEX → DENARY

3E = (3 × 16) + (14 × 1) = 48 + 14 = **62**

1A7 = (1 × 256) + (10 × 16) + 7 = 256 + 160 + 7 = **423**

DENARY → HEX

227 ÷ 16 = 14 remainder 3 → E and 3 → **E3**

500 ÷ 256 = 1 r 244; 244 ÷ 16 = 15 r 4 → **1F4**

Or convert to binary first, then group into nibbles.

Preview

PREVIEW

Why hexadecimal is used

Hexadecimal is **easier for humans** to read, write, remember and check than binary because it is a **much shorter representation of binary** (one hex digit = 4 bits). There is **less chance of error** when copying it, and it converts to binary very easily. **The computer itself still works in binary** – hex is only for people.

Where hex is used	Example
MAC addresses	`00-1A-2B-3C-4D-5E`
HTML / CSS colour codes	`#FF0000` = red (FF red, 00 green, 00 blue)
IPv6 addresses	`2001:0db8:85a3:0000:0000:8a2e:0370:7334`
Error codes / memory dumps	Windows "stop" codes such as `0x0000007B`; viewing contents of memory when debugging
Assembly code & memory addresses	Machine code and locations in RAM are shown in hex
URLs	Special characters, e.g. a space is `%20`

Adding two 8-bit binary numbers

Sum	Result	Carry
0 + 0	0	0
0 + 1	1	0
1 + 1	0	1
1 + 1 + 1	1	1

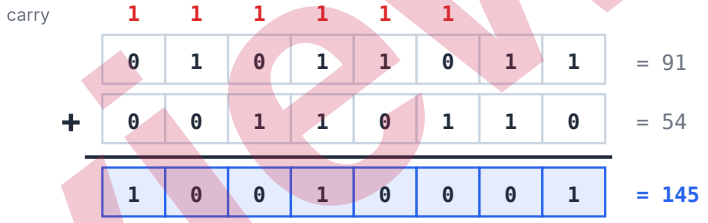


Figure 1.3 *91 + 54 = 145. Work right → left, writing carries above the next column.*

OVERFLOW

A computer or device has a **predefined limit** to the size of number it can store – for example an 8-bit register can hold at most **255** (a 16-bit register 65 535). An **overflow error** occurs when the result of a calculation is **too large to fit** in the number of bits available – the extra (9th) carry bit is lost, so the stored answer is wrong.

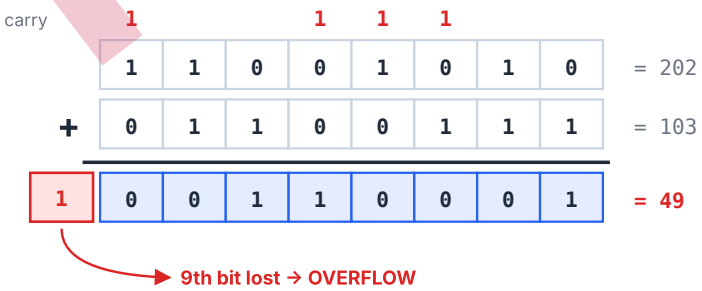


Figure 1.4 *202 + 103 = 305, which is bigger than 255. The carry out of the MSB cannot be stored, so the register holds 49 – an* **overflow error**.

Logical binary shifts

In a **logical shift** every bit moves left or right by a set number of places. **Bits shifted off the end are lost** and **zeros are shifted in** at the opposite end.

- Each **left** shift **multiplies** by 2 (left 2 = × 4, left 3 = × 8).
- Each **right** shift **divides** by 2 (whole-number division – remainders are lost).
- If a **1** is shifted off the end (the most significant bit on a left shift, or least significant bit on a right shift), the answer becomes **inaccurate**.

PREVIEW

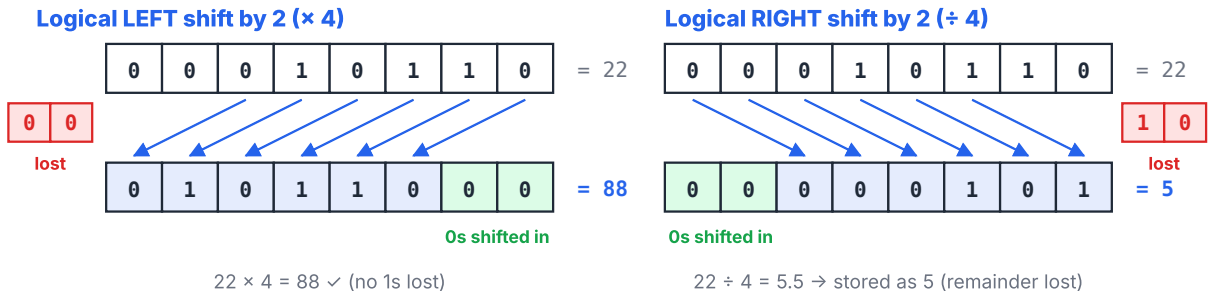


Figure 1.5 *Logical shifts. Bits falling off the end are lost; zeros fill the gap.*

COMMON MISTAKE

Don't do a *circular* shift. In a logical shift the lost bits never come back on the other side – only 0s come in.

Two's complement (negative numbers)

In 8-bit **two's complement** the most significant bit (MSB) has a **negative** place value of **−128**. All other columns are positive as normal. If the MSB is 1 the number is negative; if it is 0 the number is positive.

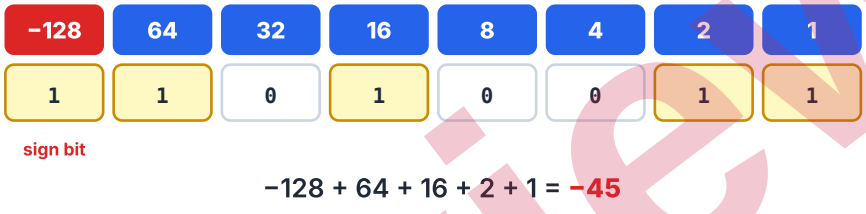


Figure 1.6 *8-bit two's complement: the MSB is worth −128.*

8-bit two's complement range: −128 to +127

(unsigned 8-bit range is 0 to 255)

DENARY → TWO'S COMPLEMENT: −45

1. Write +45 in binary: `00101101`
2. Invert (flip) every bit: `11010010`
3. Add 1: `11010011`

Check: −128 + 64 + 16 + 2 + 1 = **−45** ✓

TWO'S COMPLEMENT → DENARY: 10110110

Use the place values: −128 + 32 + 16 + 4 + 2 = **−74**

A positive two's complement number (MSB 0) is read exactly like normal binary, e.g. `01100100` = 100.

SHORTCUT (BEYOND THE SYLLABUS, BUT HANDY)

To negate a binary number quickly: starting from the **right**, copy every bit up to and including the **first 1**, then flip all the remaining bits. e.g. +44 = `00101100` → copy "100", flip the rest → `11010100` = −44.

1.2 Text, sound and images

Text and character sets

Each key on a keyboard is given a unique binary code. A **character set** is the list of all characters a computer can represent together with the binary code for each. Text is converted to binary using the character set so that it can be processed.

Preview

PREVIEW

A ASCII

- American Standard Code for Information Interchange
- Standard ASCII uses **7 bits** → 128 characters (extended ASCII uses 8 bits → 256)
- English letters, digits, punctuation and control codes
- Codes are in order: 'A' = 65, 'B' = 66 ... 'a' = 97, '0' = 48
- Uses **less storage** per character

U UNICODE

- Uses **more bits per character** (up to 32 bits / 4 bytes, typically 16 or 32)
- Represents a **far greater range** of characters – over a million possible
- Covers most world **languages** (e.g. Bangla, Arabic, Chinese) and **emojis** 😀
- The first 128 Unicode codes are the same as ASCII
- Files take **more storage space**

Sound

Sound is an **analogue** wave. To store it, the amplitude (height) of the wave is **sampled** at regular time intervals and each sample is converted into a binary value. This is done by an analogue-to-digital converter (ADC).

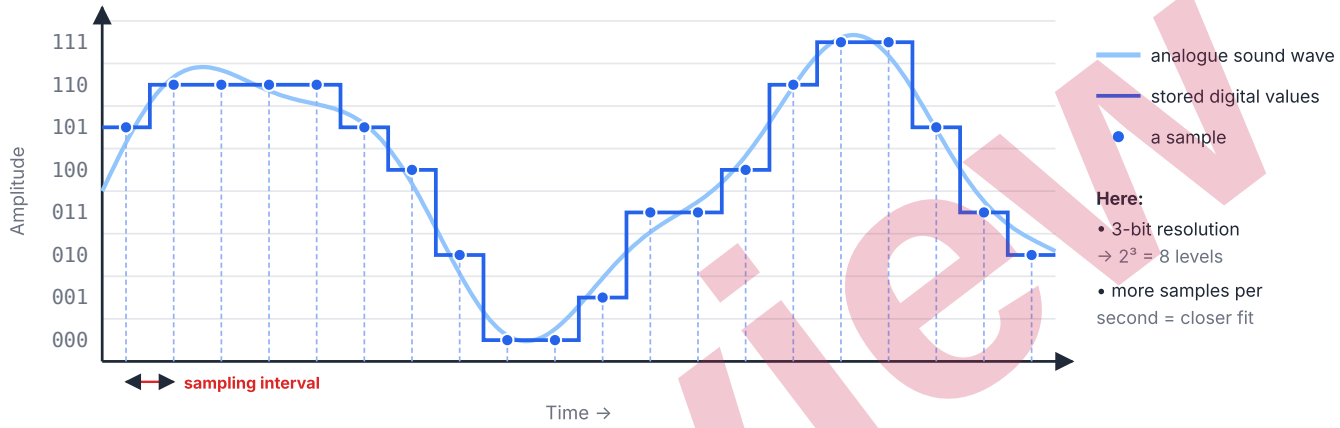


Figure 1.7 *Sampling a sound wave. Each sample's amplitude is rounded to the nearest level and stored in binary.*

Term	Meaning	Effect of increasing it
Sample rate	The number of samples taken **per second** (measured in hertz, Hz). CD quality is 44 100 Hz.	Recording more accurate / closer to the original sound; **file size increases**
Sample resolution	The number of **bits used to store each sample** (e.g. 16-bit). More bits = more possible amplitude levels.	Each sample more accurate, wider dynamic range; **file size increases**

Images (bitmap)

A bitmap image is made of a grid of tiny dots called **pixels** (picture elements). The colour of each pixel is stored as a binary code.

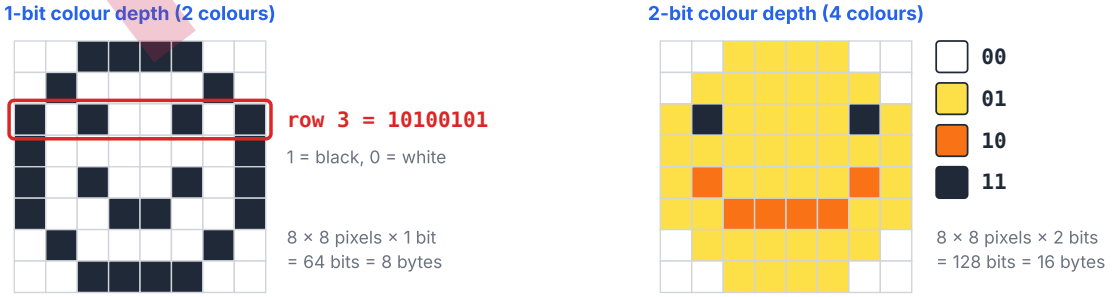


Figure 1.8 *The same 8 × 8 resolution with two colour depths. Doubling bits per pixel doubles the file size.*

Preview

PREVIEW

RESOLUTION

The **number of pixels** in the image, e.g. 1920 × 1080. Higher resolution → more detail, sharper image, larger file.

COLOUR DEPTH

The **number of bits used to represent each colour** (each pixel). *n* bits give 2^n colours: 1-bit = 2, 8-bit = 256, 24-bit ≈ 16.7 million.

As resolution and colour depth increase, the **quality** of the image increases but so does the **file size**.

1.3 Data storage and compression

Units of data storage

Cambridge uses the **binary (IEC) prefixes**, where each unit is **1024** times the previous one. Calculations must use 1024, not 1000.

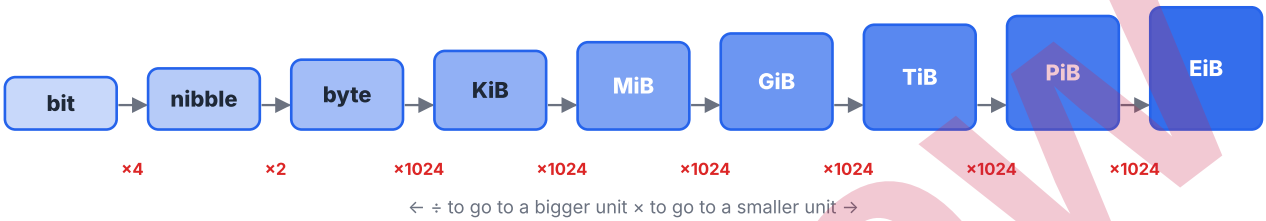


Figure 1.9 *The ladder of storage units.*

Unit	Symbol	Size	Unit	Symbol	Size
bit	b	a single 0 or 1	gibibyte	GiB	1024 MiB
nibble	–	4 bits	tebibyte	TiB	1024 GiB
byte	B	8 bits	pebibyte	PiB	1024 TiB
kibibyte	KiB	1024 bytes	exbibyte	EiB	1024 PiB
mebibyte	MiB	1024 KiB	Memory aid: **K**ind **M**onkeys **G**et **T**en **P**eanuts **E**ach		

Calculating file sizes

Image size (bits) = width × height × colour depth

width × height = resolution (number of pixels)

Sound size (bits) = sample rate × resolution × length (s)

× number of channels if stereo (2)

Then: **÷ 8** → bytes **÷ 1024** → KiB **÷ 1024** → MiB **÷ 1024** → GiB

IMAGE EXAMPLE

An image is 1024 × 512 pixels with a colour depth of 16 bits. Size in MiB?

1024 × 512 × 16 = 8 388 608 bits
÷ 8 = 1 048 576 bytes
÷ 1024 = 1024 KiB
÷ 1024 = **1 MiB**

SOUND EXAMPLE

A 60-second mono track, sample rate 16 000 Hz, resolution 16 bits. Size in KiB?

16 000 × 16 × 60 = 15 360 000 bits
÷ 8 = 1 920 000 bytes
÷ 1024 = **1875 KiB**

Preview

PREVIEW

CALCULATOR-FREE TRICKS

No calculators are allowed. Cancel early: dividing by 8 then by 1024 is the same as dividing by 8192 = 2^{13}. Look for powers of two in the numbers (512 = 2^9, 1024 = 2^{10}) and cancel them. Always **show your working** – method marks are available – and give the answer **in the unit asked for**.

Data compression

Compression reduces the size of a file. This means:

📉 **Less storage space** is needed on devices

📶 **Less bandwidth** is needed to send it

⚡ **Shorter transmission** / upload / download time

It also lets websites load faster, saves mobile data, and lets more files be stored or streamed.

	Lossy compression	Lossless compression
What it does	Reduces file size by **permanently removing data**	Reduces file size **without permanent loss of data** – the original can be fully restored
How	Reducing resolution or colour depth; reducing sample rate or resolution; removing sounds humans can't hear (perceptual music shaping)	Finds repeated patterns and stores them more efficiently, e.g. **run-length encoding (RLE)**
File size	Much smaller	Smaller, but not by as much
Use when	Small loss in quality is acceptable: photos (JPEG), music (MP3), video streaming (MP4)	Every bit must be kept: text documents, program code / executables, spreadsheets, (PNG, ZIP, FLAC)

Run-length encoding (RLE)

RLE replaces a **run** of identical, consecutive data items with **one copy of the item and a count** of how many times it repeats.



Figure 1.10 *Run-length encoding of one row of pixels. Decoding the pairs gives back exactly the original row – it is* **lossless**.

REMEMBER

RLE only works well when data has **long runs** of repeated values (e.g. simple images with large blocks of one colour). For data with few repeats, RLE can even make the file **bigger**.

Chapter 1 summary

- All data is stored as binary (on/off switches)
- Hex = shorthand for binary; 1 hex digit = 1 nibble
- 8-bit: 0–255 unsigned; −128 to +127 two's complement
- Overflow: result too big for the register
- Left shift × 2, right shift ÷ 2 per place
- Sample rate = samples per second; resolution = bits per sample
- Image resolution = pixels; colour depth = bits per pixel
- Use 1024 for all unit conversions
- Lossy = data removed for ever; lossless = fully restorable
- RLE = value + count for each run

Preview

PREVIEW

Chapter 1 · Practice Questions

Section A: quick-fire questions · Section B: O Level style structured questions · Answers at the end

SECTION A – SHORT ANSWER & MULTIPLE CHOICE

1 Which of these is the 8-bit binary for denary 99? [1]

A 01100011
B 01100101
C 11000110
D 10011001

2 Convert the hexadecimal number **B7** into (a) binary and (b) denary. [2]

3 Convert the denary number **350** into hexadecimal. Show your working. [2]

4 How many different values can be represented with 12 bits? [1]

A 12
B 1024
C 2048
D 4096

5 Give the 8-bit two's complement representation of **−100**. [1]

6 The binary number `00110100` is logically shifted **two places to the right**. Give the result and state its effect on the denary value. [2]

7 How many bits are there in 3 KiB? [1]

A 3000
B 3072
C 24 000
D 24 576

8 State **one** reason why Unicode is used instead of ASCII. [1]

9 Encode the following pixel row with run-length encoding: `R R R R R G G B B B B B B B` [2]

10 Which statement about lossy compression is **true**? [1]

A The original file can be fully restored
B It is used for program files
C It permanently removes data
D It always uses RLE

SECTION B – STRUCTURED QUESTIONS

Question 11

A computer stores numbers in 8-bit registers.

(a) Add the binary numbers `10011110` and `01110011`. Show your working. [2]

(b) Explain why the result in (a) causes an error. [2]

(c) Register X contains `00011011`. A logical left shift of three places is performed. Give the new contents and explain the effect on the value. [3]

(d) Convert the two's complement number `11100101` to denary. [1]

Question 12

Hexadecimal is often used in computer science.

(a) Give **two** reasons why programmers use hexadecimal instead of binary. [2]

(b) Identify **two** places where hexadecimal is used. [2]

(c) An HTML colour code is `#2A9F0C`. Convert the red component `2A` into 8-bit binary. [1]

Preview

PREVIEW

Question 13

Jamila records a song and takes a photograph.

(a) Explain how sound is converted into binary so that it can be stored. Use the terms *sample rate* and *sample resolution*. 4

(b) The photograph is 2048 pixels wide and 1024 pixels high, with a colour depth of 8 bits. Calculate its file size in **MiB**. Show your working. 3

(c) Jamila wants to email the photograph. Explain why she might compress it with **lossy** compression, and give one drawback. 3

Answers – Chapter 1

1 **A** – 64 + 32 + 2 + 1 = 99.

2 (a) B = 1011, 7 = 0111 → **10110111** (b) 11 × 16 + 7 = **183**

3 350 ÷ 256 = 1 r 94; 94 ÷ 16 = 5 r 14 (E) → **15E**. (Check: 256 + 80 + 14 = 350.)

4 **D** – 2^{12} = 4096.

5 +100 = 01100100 → invert 10011011 → +1 = **10011100** (check −128 + 16 + 8 + 4 = −100).

6 **00001101**; the value is **divided by 4** (52 → 13). 1 mark result, 1 mark effect.

7 **D** – 3 × 1024 × 8 = 24 576.

8 Any one: represents a greater range of characters; includes characters from other languages (e.g. Bangla, Chinese); includes emojis/symbols.

9 **5R 2G 7B** (or R5 G2 B7). 1 mark for correct counts, 1 for correct format/order.

10 **C**

11

- (a) 10011110 (158) + 01110011 (115) = **1 00010001**; the 8-bit register holds **00010001** with a carry of 1 out of the MSB. 1 working/carries, 1 answer
- (b) The real answer (273) is greater than 255 / needs 9 bits; the register only has 8 bits so an **overflow error** occurs – the 9th bit is lost and the stored value (17) is wrong.
- (c) **11011000** (1 mark). The value is multiplied by 2^3 = 8 (1 mark): 27 → 216 (1 mark). No 1s were lost, so the result is accurate.
- (d) −128 + 64 + 32 + 4 + 1 = **−27**

12

- (a) Any two: shorter than binary; easier for humans to read/write/remember; fewer errors when copying; easy to convert to and from binary (1 digit = 4 bits).
- (b) Any two: MAC addresses; HTML colour codes; IPv6 addresses; error codes; memory dumps / debugging; assembly language/memory addresses; URLs.
- (c) 2 = 0010, A = 1010 → **00101010**

13

- (a) Sound is an analogue wave; its amplitude is **sampled** at regular time intervals; each sample is converted to a binary number (by an ADC); the **sample rate** is the number of samples taken per second; the **sample resolution** is the number of bits used to store each sample; higher rate/resolution → more accurate recording but larger file. Any 4 points.
- (b) 2048 × 1024 × 8 = 16 777 216 bits; ÷ 8 = 2 097 152 bytes; ÷ 1024 = 2048 KiB; ÷ 1024 = **2 MiB**. 1 multiply, 1 conversion, 1 answer.
- (c) Lossy gives a much smaller file → quicker to upload/send, less bandwidth, may fit inside email attachment limit (2 marks). Drawback: data is permanently removed so image quality is reduced / original cannot be restored (1 mark).

Preview

PREVIEW

CHAPTER 2 · SYLLABUS TOPIC 2

Data Transmission

Paper 1 · Computer Systems

By the end of this chapter you can:

- Describe the structure of a packet
- Describe packet switching step by step
- Compare serial and parallel transmission
- Compare simplex, half-duplex and full-duplex
- Choose a method for a given scenario
- Explain how USB transmits data
- Explain why errors occur in transmission
- Use parity, checksum and echo checks
- Calculate a check digit (ISBN, barcode)
- Describe ARQ and symmetric/asymmetric encryption

2.1 Types and methods of data transmission

Packets

When data (e.g. a file, email or web page) is sent across a network it is **broken down into small, equal-sized units called packets**. Small packets can be sent independently and quickly; if one is lost or damaged only that packet needs to be sent again.

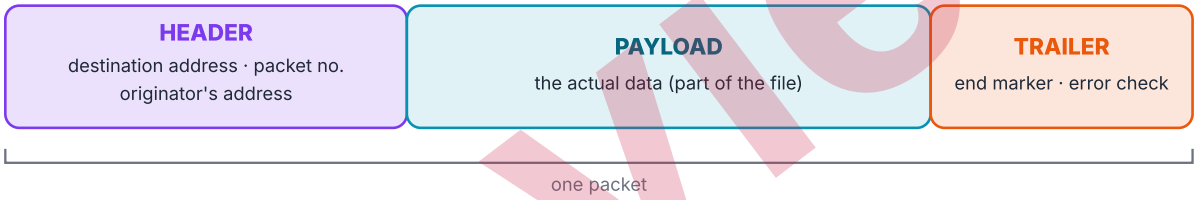


Figure 2.1 *The structure of a data packet.*

Part	Contents
Packet header	• **Destination address** (IP address of the receiving device) • **Packet number** (so packets can be put back in the correct order) • **Originator's address** (IP address of the sender) Often also: total number of packets, size of packet.
Payload	The **actual data** being carried (part of the file). Typically about 64 KiB or less.
Trailer	Marks the **end of the packet** and contains an **error-checking** value (e.g. a checksum / cyclic redundancy check) so the receiver can tell if the packet has been corrupted.

Preview

PREVIEW

Packet switching

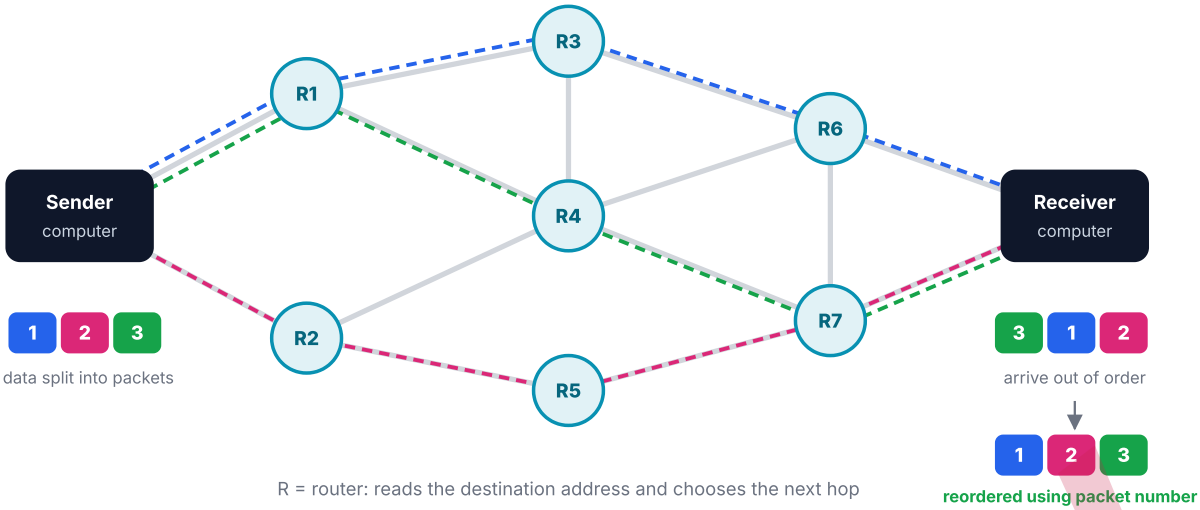


Figure 2.2 *Packet switching. Packets of one file travel by different routes and are reassembled at the destination.*

1. The data is **broken down into packets**; each is given a header with the destination address, packet number and originator's address.
2. Each packet is sent onto the network. **Each packet could take a different route** to the destination.
3. **Routers** examine the destination address in each packet header and **control the route** each packet takes, sending it along the best/least busy path available at that moment.
4. Because routes differ in length and traffic, **packets may arrive out of order**.
5. Once the **last packet has arrived**, the receiving device uses the packet numbers to **reorder** the packets and reassemble the original data. Missing packets can be requested again.

ADVANTAGES

- Packets avoid busy or broken parts of the network – very reliable
- Network links are shared efficiently by many users
- Only lost/corrupt packets need resending, not the whole file
- Faster overall: packets travel at the same time on different routes

DISADVANTAGES

- Packets must be reordered at the destination (delay)
- Packets can be lost and need re-sending
- Delays can be a problem for real-time data such as video calls
- Headers add extra data to each packet

Serial and parallel transmission

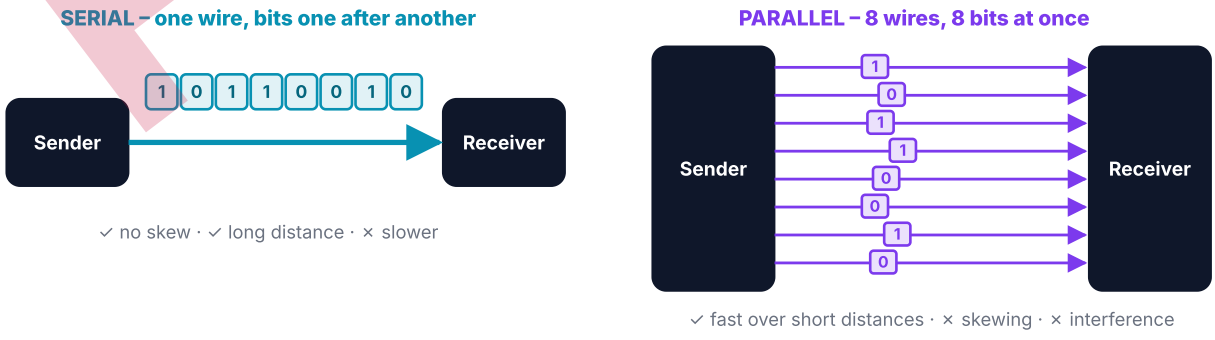


Figure 2.3 *Serial vs parallel. In parallel, bits on different wires can drift apart (**skew**) over long distances.*

	Serial	Parallel
How	Data is sent **one bit at a time** down a **single wire** / channel	**Several bits** (e.g. 8) are sent **at the same time** down **several wires**

Preview